ShelfSight — Meta Global AI Developer Hackathon Application

Oct 24, 2026 · @richi_mc

At a glance

ShelfSight turns one ordinary camera into an automatic inventory clerk for small stores: it watches the shelves and records every product taken out or restocked, with no barcodes, no RFID tags and no model training per product.

Field	Answer
Project	ShelfSight: automatic shelf inventory from video
One-line pitch	A camera that knows what left your shelf, so a corner store always knows its stock
Track	Indie developer (solo student)
Theme	Build for Your World
Builder	José Ricardo Mendoza Castañeda, student at ESCOM, Instituto Politécnico Nacional (Mexico City)
Meta tools	SAM 3.1 and Muse Spark on Meta Model API; Muse Code to build; SAM 3.1 open weights as a GPU fallback
Demo on Nov 4	A real shelf with 10–15 products and a live camera; the dashboard updates stock as people take and restock items

Problem

Mexico has more than 1.05 million tienditas (corner stores) serving about 52% of national consumer demand (Milenio, Aug 2024, citing a Bit by Teamcore report), yet most of them cannot tell you their stock right now.

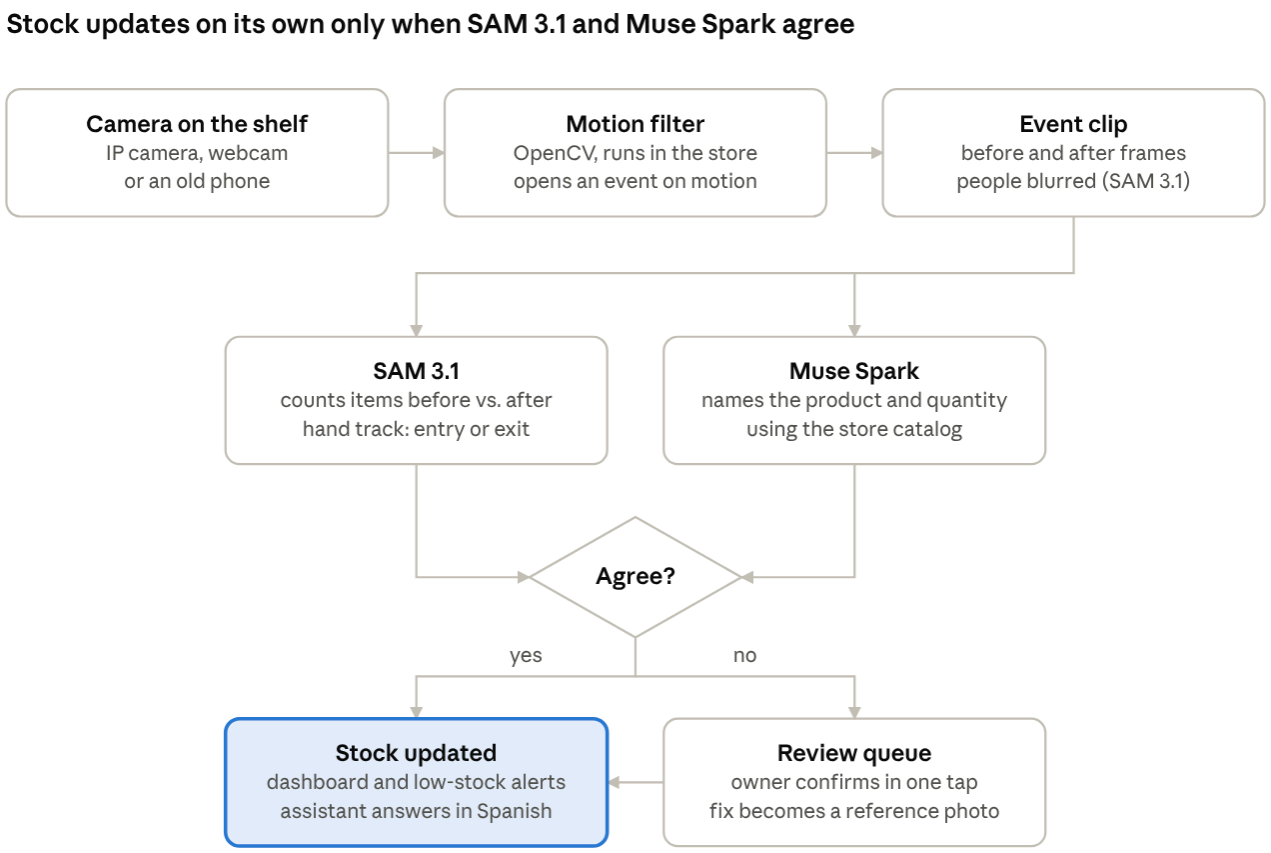
- **Counting is manual.** The owner walks the shelves, usually after closing, and still finds out a product ran out when a customer asks for it.

- **Point-of-sale systems assume every sale is scanned.** Many small stores have no scanner at all, so the system never learns what left the shelf and the numbers drift.
- **Existing retail vision is built for chains.** Cashierless-store systems need many ceiling cameras and models trained per product: the cost and setup of a supermarket, not of a one-person store.
- **The cost is real:** lost sales from empty shelves, over-ordering from distributors, and no visibility into products that disappear.

I work with small-business data in my thesis, Negocio Seguro, which predicts whether a new small business in Mexico City will survive. ShelfSight attacks the daily operation side of the same problem.

How it works

ShelfSight treats every interaction with a shelf as an event, and turns each event into a stock movement such as "−2 Coca-Cola 600 ml, shelf 3, 14:32" (exit) or "+12 Sabritas, restock" (entry).



ShelfSight event pipeline · 7 boxes, 1 decision

Both models look at the same event; a disagreement never changes stock, it goes to the owner's review queue instead.

1. **Setup in minutes.** The owner photographs each shelf once. Muse Spark identifies the products and drafts the store catalog; SAM 3.1 segments every item to set the starting count per shelf zone. The owner only confirms names and prices.
2. **Capture.** Any IP camera, webcam or old phone points at the shelves. A light OpenCV motion filter runs locally and opens an event only when something enters a shelf zone, so the models are never called on empty video.
3. **Event clip.** Each event keeps a short clip plus a "before" and an "after" frame of the shelf. People are segmented with SAM 3.1 and blurred before any frame leaves the store.
4. **Count the change (SAM 3.1).** Short text prompts ("bottle", "chip bag", "can") segment and count products in the before and after frames, and tracking the hand and the object in the clip gives the direction: into the shelf (entry) or out of it (exit).
5. **Identify the product (Muse Spark).** The clip and crops of the changed regions go to Muse Spark with the catalog and reference photos in context. It returns structured JSON: action (removed, added, moved, none), product, quantity, confidence.
6. **Reconcile.** A rules layer compares both signals. When SAM 3.1's count change and Muse Spark's answer agree, stock updates automatically. When they disagree or confidence is low, the event goes to a review queue with its clip, and the owner fixes it with one tap. Each correction is added to the catalog as a new reference photo, so the same mix-up is less likely next time.
7. **Act on it.** A dashboard shows live stock per product and shelf, an event log with clips, and low-stock alerts. The owner can also ask in Spanish, "¿cuántas Coca-Colas me quedan y cuándo pido más?", and a Muse Spark agent answers by calling the inventory API.

How Meta's models are used

Two Meta models split the work: SAM 3.1 answers "how many and which way", Muse Spark answers "which product", and agreement between them is what lets stock update without a human.

Meta model or tool	Role in ShelfSight	Why it fits
SAM 3.1 (Meta Model API)	Segment, count and track products and hands; blur people	Open-vocabulary text prompts plus boxes and points, so no training per product; joint multi-object tracking keeps a crowded shelf fast

Meta model or tool	Role in ShelfSight	Why it fits
Muse Spark (Meta Model API)	Identify the product and quantity per event; draft the catalog from shelf photos; answer the owner through tool calls	Native image and video input, 1M-token context that fits the whole catalog with reference photos, OpenAI-compatible API
Muse Code	Coding agent for the pipeline, API and dashboard	Lets one builder ship a full-stack system in 10 days
SAM 3.1 open weights (Modal or Baseten GPU)	Fallback if an API limit or a missing video feature blocks the demo	Keeps the demo running on the same model

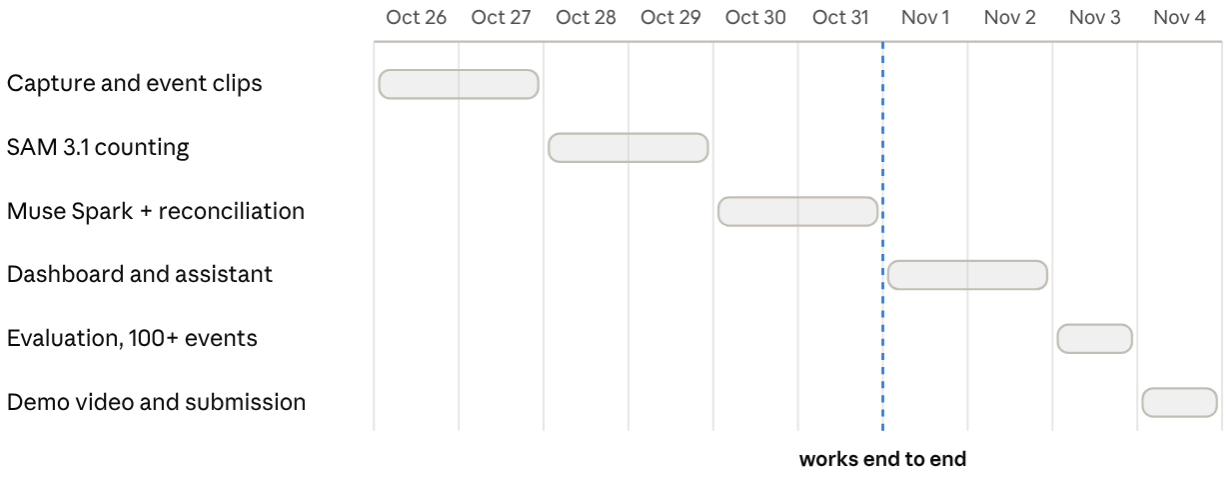
Privacy by design. ShelfSight counts products, not people. Cameras frame the shelves, people are blurred with SAM 3.1 before frames reach any API, there is no face recognition, and clips are kept only for events under review.

Feasibility: 10-day plan, metrics and risks

The MVP fits ten days because no model is trained: the custom code is capture, the fusion rules, the inventory API and the dashboard, and the end-to-end path works by day 6 so the last days go to accuracy.

The pipeline works end to end by Oct 31, day 6 of 10

Build plan by day, Oct 26 to Nov 4, 2026; submissions due Nov 4 at 9:00 PM ET



10-day build plan · Oct 26 to Nov 4, 2026

Days 7 to 10 add the owner-facing app, measure accuracy on recorded events and produce the demo video.

Success targets, measured on a recorded test set of at least 100 staged events on a real shelf:

Metric	Target
Shelf interactions detected	⩾ 95%
Events resolved automatically with the right product, direction and quantity	⩾ 90% (the rest go to the review queue, not into wrong stock)
Time from event to updated stock	< 15 s
Setup time for a 15-product shelf	< 5 min

Risks and mitigations

Risk	Mitigation
Hand or body hides the products	Count on the before and after frames once the hand has left, not on mid-motion frames
Look-alike products (600 ml vs 1 L)	Reference photos in the catalog, shelf-zone priors and the review queue
API latency or limits on video	Send keyframes and crops instead of full clips; SAM 3.1 open weights on a GPU as fallback
Lighting changes	Event only when the change persists after the motion ends

Technical skill: an AI project I built

I lead the backend and database of Negocio Seguro, my team's undergraduate thesis at ESCOM–IPN: a platform that predicts the probability that a new small business survives at a chosen location in Mexico City's metro area, and explains why.

- **Data and geospatial engineering.** INEGI's national business directory (DENUE) and census block data (AGEB) in PostgreSQL + PostGIS, across the 84 municipalities of the metro area. Features are computed in SQL: competitors of the same activity code within 500 m (ST_DWithin in UTM zone 14N), distance to anchors such as transit stations and markets, and census variables such as population, schooling and internet access. GIST indexes target 500 ms per geospatial query.

- **Model serving and explainability.** The API calls our ML microservice (FastAPI) and returns the survival probability, a risk level and per-factor SHAP impacts, so a non-technical owner understands the result. Each prediction stores the model version and data edition for traceability.
- **API engineering.** Bun + ElysiaJS + TypeORM, schema validation with TypeBox, auto-generated OpenAPI, JWT auth, argon2id password hashing and rate limits; 12-table schema and 25 endpoints designed.
- **Status.** Database implemented; API in implementation.

I also build native Android apps in Kotlin, which I can reuse to turn an old phone into a ShelfSight camera. ShelfSight reuses the same pattern: turn messy real-world signals into structured records, serve AI results through a reliable API, and explain them to small-business owners.

Ready to paste into the form

Use the version that fits each field's limit; the reviewers score only the form, on idea clarity and fit, and on technical skill, weighted equally.

Tagline (92 characters)

ShelfSight: a camera that knows what left your shelf, so every corner store knows its stock.

Project idea, short (about 50 words)

ShelfSight turns one cheap camera into an automatic inventory clerk for Mexico's 1M+ corner stores. SAM 3.1 counts and tracks products entering or leaving each shelf; Muse Spark identifies which product moved. When both agree, stock updates instantly; when they don't, the owner confirms with one tap. No barcodes, no training.

Project idea, full (about 200 words)

Mexico's 1.05 million tienditas serve about half of national consumer demand, yet most track inventory by hand: the owner counts shelves after closing and learns a product ran out when a customer asks for it. Barcode POS systems assume every sale is scanned, and cashierless-store vision needs many cameras and models trained per product.

ShelfSight needs one ordinary camera and no training. A local motion filter opens an event when a hand reaches a shelf. SAM 3.1 segments and counts products in the before and after frames and tracks the hand to tell an exit (sale) from an entry (restock). Muse Spark receives the clip, the changed regions and the store catalog, and returns which product moved and how many as structured JSON. When the two signals agree, stock updates automatically; when they don't, the event goes to a one-tap review queue, and each

correction becomes a new reference photo. Owners get live stock, low-stock alerts and a Spanish chat agent that answers by calling the inventory API. People are blurred before frames leave the store.

Demo on Nov 4: a real 15-product shelf, a live camera and a measured accuracy report on 100+ staged events.

Theme fit: Build for Your World (about 50 words)

I study small-business survival in Mexico City through my thesis. ShelfSight brings Meta's models to a problem corner-store owners face every day, frees the hours they spend counting by hand, and makes possible something that was out of their reach: real-time inventory without new hardware or technical skills.

How I will use Meta's models (about 75 words)

SAM 3.1 (Meta Model API) segments, counts and tracks products and hands with short text prompts, and blurs people for privacy. Muse Spark (Meta Model API) uses its image and video input and 1M-token context to identify the product and quantity in each event, drafts the catalog from shelf photos, and powers a tool-calling assistant over the inventory API. Muse Code accelerates the build; SAM 3.1 open weights on a GPU are the fallback.

10-day plan (one line)

Days 1–2 capture and event clips; 3–4 SAM 3.1 counting and direction; 5–6 Muse Spark identification and reconciliation, working end to end; 7–8 dashboard, alerts and Spanish assistant; 9 evaluation on 100+ events; 10 demo video and submission.

Sources

- Meta Global AI Developer Hackathon, event page: tracks, themes, dates and how applications are scored
- Meta Model API, Models: Muse Spark inputs (text, image, video), tool calling and structured output; SAM 3.1 on the API
- SAM 3.1 vs SAM 3, Labellerr: Object Multiplex tracking and the open-weights checkpoint
- Milenio, Aug 13, 2024: number of tienditas and share of demand